



LECTURE 03

Introduction to Database Systems

Introduction to SQL

Xiaofan Li

School of Computer Science, Nanjing University



Learning Objectives

01

Define and modify tables using domain types, integrity constraints, and schema-update commands

02

Construct and refine SELECT-FROM-WHERE queries, joins, predicates, ordering, and duplicate handling

03

Apply set operations, reason about NULL and UNKNOWN, and aggregate values with GROUP BY and HAVING



Lecture Outline

01

Data definition and table updates

02

Query structure and refinements

03

Set operations, NULL, and aggregation



Data Definition Language



Data Definition Language (DDL)

The data-definition language specifies **information about relations**:

Core definitions

- The schema of each relation
- Attribute domains
- Integrity constraints

Additional definitions

- Indexes
- Security and authorization
- Physical storage information



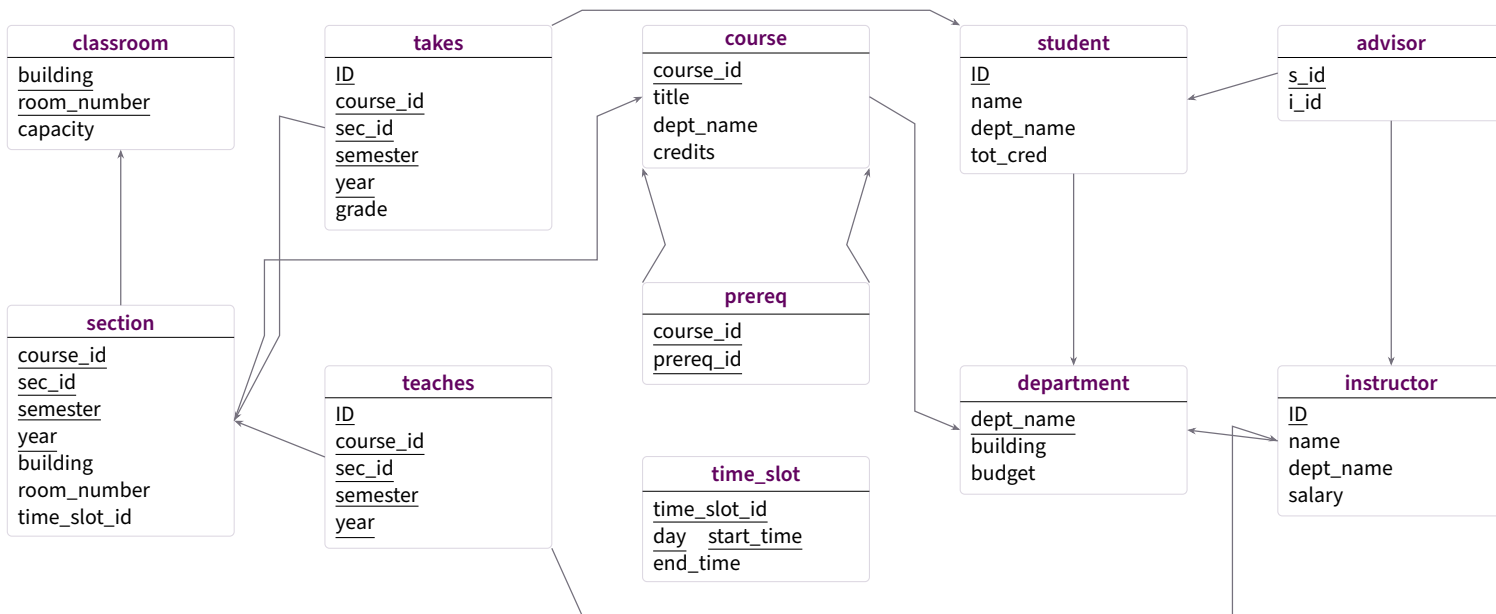
Domain Types in SQL

Type	Meaning
<code>char(n)</code>	Fixed-length character string, padded when shorter than n
<code>varchar(n)</code>	Variable-length string with maximum length n
<code>int</code> , <code>smallint</code>	Integer types with implementation-dependent ranges
<code>numeric(p,d)</code>	Exact decimal: p total digits, d after the decimal point; <code>numeric(3,1)</code> stores 44.5 exactly, but not 444.5 or 0.32 exactly
<code>real</code> , <code>double precision</code>	Floating-point numbers with implementation-dependent precision
<code>float(n)</code>	Floating-point type with requested precision; interpretation is system-dependent

Further types are discussed in Chapter 4.



University Database: Schema Diagram





The CREATE TABLE Construct

Define a relation by giving its name, attributes, types and constraints.

```
CREATE TABLE r (  
  A1 D1,  
  A2 D2,  
  ...,  
  An Dn,  
  integrity_constraint1,  
  ..., integrity_constraintk  
);
```

r : relation name

A_i : attribute name

D_i : domain type for A_i



Creating the Instructor Table

```
CREATE TABLE instructor (  
    ID char(5),  
    name varchar(20),  
    dept_name varchar(20),  
    salary numeric(8, 2)  
);
```



CREATE TABLE: Integrity Constraints

NOT NULL PRIMARY KEY FOREIGN KEY ... REFERENCES

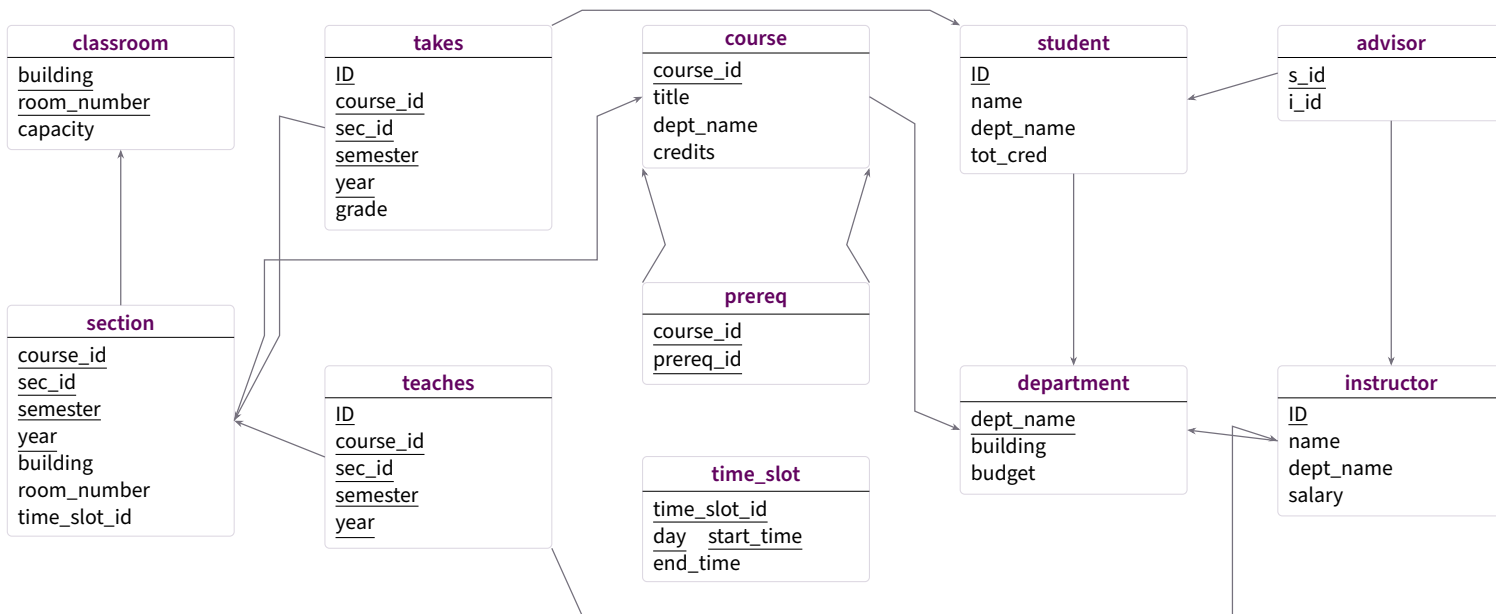
```
CREATE TABLE instructor (  
  ID char(5),  
  name varchar(20),  
  dept_name varchar(20),  
  salary numeric(8, 2)  
);
```

```
CREATE TABLE instructor (  
  ID char(5),  
  name varchar(20) NOT NULL,  
  dept_name varchar(20),  
  salary numeric(8, 2),  
  PRIMARY KEY (ID),  
  FOREIGN KEY (dept_name)  
    REFERENCES department (dept_name)  
);
```

A primary key is both **unique** and **not null**.



University Schema: Relations to Define





CREATE TABLE: More Relations

```
CREATE TABLE student (  
  ID varchar(5),  
  name varchar(20) NOT NULL,  
  dept_name varchar(20),  
  tot_cred numeric(3, 0),  
  PRIMARY KEY (ID),  
  FOREIGN KEY (dept_name)  
    REFERENCES department  
);
```

```
CREATE TABLE course (  
  course_id varchar(8),  
  title varchar(50),  
  dept_name varchar(20),  
  credits numeric(2, 0),  
  PRIMARY KEY (course_id),  
  FOREIGN KEY (dept_name)  
    REFERENCES department  
);
```

```
CREATE TABLE takes (  
  ID varchar(5),  
  course_id varchar(8), sec_id varchar(8),  
  semester varchar(6), year numeric(4, 0),  
  grade varchar(2),  
  PRIMARY KEY (ID, course_id, sec_id,  
               semester, year),  
  FOREIGN KEY (ID) REFERENCES student,  
  FOREIGN KEY (course_id, sec_id, semester, year)  
    REFERENCES section  
);
```

Example: two sections of one course

Student 12345 takes sections 1 and 2 of CS-101 in Fall 2026.

With sec_id: the keys differ at 1 versus 2; both rows are valid.

Without sec_id: both keys become (12345, CS-101, Fall, 2026); the second row is rejected.



Updating Tables and Schemas

INSERT

DML

```
INSERT INTO instructor  
VALUES ('10211', 'Smith', 'Biology', 66000);
```

DELETE

DML

```
DELETE FROM student;  
Remove all rows; keep the table.
```

DROP

DDL

```
DROP TABLE r; Remove the table itself.
```

ALTER

DDL

```
ALTER TABLE r ADD A D;  
ALTER TABLE r DROP COLUMN A;  
New nullable columns without a default have null values in existing rows.
```



Basic Query Structure



Basic Query Structure

```
SELECT A1, A2, ..., An  
FROM r1, r2, ..., rm  
WHERE P;
```

A_i : selected attributes

r_i : input relations

P : predicate

The query result is a **table**.



The SELECT Clause

SELECT lists the **attributes wanted** in the output. It corresponds to **projection** in relational algebra.

```
SELECT name  
FROM instructor;
```

SQL keywords and ordinary, unquoted identifiers are generally case-insensitive: name = NAME = Name.



SELECT: Duplicates

SQL permits duplicates in tables and query results.

DISTINCT Remove duplicates

```
SELECT DISTINCT dept_name  
FROM instructor;
```

ALL Retain duplicates

```
SELECT ALL dept_name  
FROM instructor;
```

ALL is the default.



SELECT: All Attributes and Literals

All attributes

```
SELECT * FROM instructor;
```

Literal without FROM

```
SELECT '437';  
SELECT '437' AS F00;
```



FOO
437

Both return one row; AS F00 supplies the column heading.

Literal with FROM

```
SELECT 'A' FROM instructor;
```



result
A
A
⋮
A

Repeated once per input row: 12 rows in the sample table.



SELECT: Arithmetic Expressions

Expressions can use constants, attributes and the operators $+$, $-$, $*$ and $/$.

```
SELECT ID, name, salary / 12  
FROM instructor;
```

Give the computed column a name with **AS**:

```
SELECT ID, name, salary / 12 AS monthly_salary  
FROM instructor;
```



The WHERE Clause

WHERE filters rows using a predicate, corresponding to **relational-algebra selection**.

```
SELECT name FROM instructor  
WHERE dept_name = 'Comp. Sci.';
```

Combine comparisons using AND, OR and NOT:

```
SELECT name FROM instructor  
WHERE dept_name = 'Comp. Sci.' AND salary > 80000;
```

Additional Query Operations

Combining Tables

Products and Joins

Connect related rows using Cartesian products, joins, aliases, and self-joins.



Cartesian Product

Find the Cartesian product `instructor` $\times$ `teaches`.

```
SELECT *  
FROM instructor, teaches;
```

Every `instructor`–`teaches` pair appears, with all attributes from both relations. Both relations contain an attribute named `ID`; write `instructor.ID` or `teaches.ID` to state which relation's `ID` is intended.

A `WHERE` condition then selects the meaningful combinations.



Cartesian Product: Worked Tables

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

teaches

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

instructor × teaches

I.ID	name	dept.	salary	T.ID	course	sec.	sem.	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2009
...								
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2009
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2010
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2009
...								

I.ID = instructor.ID T.ID = teaches.ID



Joining Instructors and Teaches

Find the names of instructors who have taught a course, together with its course ID.

RELATION SCHEMAS

```
instructor(ID, name, dept_name, salary)
teaches(ID, course_id, sec_id, semester, year)
```

```
SELECT name, course_id
FROM instructor, teaches
WHERE instructor.ID = teaches.ID;
```

STEP 2: TEST THE TWO IDS

instructor.ID	teaches.ID	Keep?
10101	10101	yes
10101	12121	no
12121	10101	no
12121	12121	yes

Equijoin = equality join

1. Conceptually pair rows from both tables.
2. Keep pairs whose two ID values are equal.
3. Display only name and course_id.

STEP 3: OUTPUT COLUMNS

name	course_id
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
...	



Adding a Department Condition

Find the names and course IDs for instructors in the **Art** department who have taught a course.

```
SELECT name, course_id
FROM instructor, teaches
WHERE instructor.ID = teaches.ID
      AND instructor.dept_name = 'Art';
```



Renaming with AS

Rename a relation or an attribute: old_name AS new_name.

Find instructors earning more than **some** instructor in Comp. Sci.

```
SELECT DISTINCT T.name  
FROM instructor AS T, instructor AS S  
WHERE T.salary > S.salary AND S.dept_name = 'Comp. Sci.';
```

Table aliases distinguish two roles of the same table. instructor AS T $\equiv$ instructor T



Self-Join Example: Supervisors

`emp_super(person, supervisor)`

emp = employee; **super** = supervisor

Row (p, s) means employee p reports directly to supervisor s .

person	supervisor
Bob	Alice
Mary	Susan
Alice	David
David	Mary

`SELECT supervisor`

`FROM emp_super`

`WHERE person = 'Bob';`

Bob's direct supervisor is **Alice**.

SUPERVISOR OF SUPERVISOR: DAVID

`SELECT B.supervisor`

`FROM emp_super A JOIN emp_super B`

`ON A.supervisor = B.person`

`WHERE A.person = 'Bob';`

JOIN RESULT (BEFORE WHERE)

A.person	A.supervisor	B.person	B.supervisor
Bob	Alice	Alice	David
Alice	David	David	Mary
David	Mary	Mary	Susan

WHERE FILTER

`A.person = 'Bob'`

`SELECT B.SUPERVISOR`

David



Homework: All Indirect Supervisors

TASK

Using `emp_super(person, supervisor)`, write SQL that returns **all indirect supervisors of Bob**. Exclude Bob's direct supervisor. Assume the hierarchy has no cycles.

MODEL ANSWER

```
WITH RECURSIVE supervisor_chain(supervisor, depth) AS (  
    SELECT supervisor, 1  
    FROM emp_super  
    WHERE person = 'Bob'  
    UNION ALL  
    SELECT e.supervisor, c.depth + 1  
    FROM supervisor_chain AS c  
    JOIN emp_super AS e ON e.person = c.supervisor  
)  
SELECT supervisor AS indirect_supervisor  
FROM supervisor_chain  
WHERE depth >= 2;
```

HOW IT WORKS

Base case: Alice at depth 1.

Recursive step: follow each supervisor to the next supervisor.

Keep: rows with depth ≥ 2 .

EXPECTED RESULT

indirect_supervisor
David
Mary
Susan

Filtering and Presenting Results

Match strings, sort and limit output, and express conditions with BETWEEN and IN.



String Operations: LIKE

LIKE matches **string patterns**: % means any substring (including empty); _ means exactly one character.

```
SELECT name FROM instructor  
WHERE name LIKE '%dar%';
```

With case-sensitive matching: dar, darwin, sardar.

A literal percent sign: LIKE '100\%' ESCAPE '\'
'%dar___' requires exactly three characters after dar.



String Operations: More Pattern Examples

Pattern case sensitivity depends on the system and collation.

Pattern	Meaning
'Intro%'	Begins with Intro
'%Comp%'	Contains Comp
'___'	Exactly three characters
'___%'	At least three characters

Other string operations include concatenation (||), case conversion, string length and substring extraction.



Ordering the Display of Tuples

List distinct instructor names in alphabetical order.

```
SELECT DISTINCT name  
FROM instructor  
ORDER BY name;
```

ASC is the default; DESC requests descending order.

ORDER BY name DESC

Sort on multiple attributes: ORDER BY dept_name, name.



Limiting the Number of Output Rows

Return part of a large result. Syntax differs across database systems.

SQL Server: TOP

```
SELECT DISTINCT TOP (10) name  
FROM instructor  
ORDER BY name;
```

Standard-style FETCH

```
SELECT DISTINCT name  
FROM instructor  
ORDER BY name  
FETCH FIRST 10 ROWS ONLY;
```

MySQL uses LIMIT; Oracle supports FETCH and ROWNUM. Use ORDER BY for a predictable first set of rows.



WHERE Predicates: Ranges and Tuple Comparison

BETWEEN includes both endpoints: $90000 \leq \textit{salary} \leq 100000$.

```
SELECT name FROM instructor
WHERE salary BETWEEN 90000 AND 100000;
```

Tuple comparison combines matching conditions:

```
SELECT name, course_id FROM instructor, teaches
WHERE (instructor.ID, dept_name) = (teaches.ID, 'Biology');
```



The IN Operator

IN tests **membership** in a list of values. It is a compact alternative to multiple OR conditions.

```
SELECT name  
FROM instructor  
WHERE dept_name IN ('Comp. Sci.', 'Biology');
```

Understanding Duplicates

Multiset Semantics

Track repeated rows through selection, projection, and Cartesian products.



Duplicates: Multiset Semantics in RA

A multiset retains the number of copies of each tuple.

Selection

If tuple t_1 occurs c_1 times in r_1 and satisfies θ , all c_1 copies appear in $\sigma_\theta(r_1)$.

Projection

Every input copy contributes one projected copy. Equal projected values remain duplicated.

Product

If t_1 occurs c_1 times and t_2 occurs c_2 times, their combined tuple occurs $c_1 c_2$ times.



Duplicates: Worked Example

$$r_1(A, B) = \{(1, a), (2, a)\} \quad r_2(C) = \{(2), (3), (3)\}$$

$$\pi_B(r_1) = \{(a), (a)\} \quad \text{under multiset semantics}$$

$$\pi_B(r_1) \times r_2 = \{(a, 2), (a, 2), (a, 3), (a, 3), (a, 3), (a, 3)\}$$

```
SELECT A1, A2, ..., An  
FROM r1, r2, ..., rm  
WHERE P;
```

Multiset interpretation:

$$\pi_{A_1, A_2, \dots, A_n}(\sigma_P(r_1 \times r_2 \times \dots \times r_m))$$

Combining Query Results

Set Operations

Combine result sets with UNION, INTERSECT, and EXCEPT; control duplicates with ALL.



Set Operations: UNION, INTERSECT, EXCEPT

The two query results must be **union-compatible**: the same number of columns, in the same order, with compatible types.

UNION

```
SELECT course_id
FROM section
WHERE semester='Fall'
AND year=2009
UNION
SELECT course_id
FROM section
WHERE semester='Spring'
AND year=2010;
```

Either term

INTERSECT

```
SELECT course_id
FROM section
WHERE semester='Fall'
AND year=2009
INTERSECT
SELECT course_id
FROM section
WHERE semester='Spring'
AND year=2010;
```

Both terms

EXCEPT

```
SELECT course_id
FROM section
WHERE semester='Fall'
AND year=2009
EXCEPT
SELECT course_id
FROM section
WHERE semester='Spring'
AND year=2010;
```

Fall only



Set Difference: Finding the Maximum

A salary is **not Max** if it is smaller than some other salary. Remove all such values from the known salaries.

```
SELECT DISTINCT salary
FROM instructor
WHERE salary IS NOT NULL
EXCEPT
SELECT DISTINCT T.salary
FROM instructor AS T,
      instructor AS S
WHERE T.salary < S.salary;
```

With at least one known salary, this query returns its maximum value.



Set Operations with Duplicates

Default set operations

UNION
INTERSECT
EXCEPT

Duplicate rows are eliminated from the result.

Add **ALL** to retain bag multiplicities. If tuple t occurs m times in r and n times in s :

$r \text{ UNION ALL } s$ $m + n$ copies

$r \text{ INTERSECT ALL } s$ $\min(m, n)$

$r \text{ EXCEPT ALL } s$ $\max(0, m - n)$

Missing Information

NULL and Three-Valued Logic

Understand missing values, UNKNOWN, and how WHERE handles them.



Why NULL Values Exist

01

Not applicable A student may not have a passport.

02

Not yet known Total credits may not have been recorded when the student joins.

03

Schema evolution A newly added column may have no historical value for existing rows.

NULL marks missing information; it has different semantics from 0 or an empty string. A suitable DEFAULT may supply a value.



Working with NULL

NULL is a marker for a missing value, not an ordinary value. Arithmetic involving NULL normally produces NULL: $5 + \text{NULL} \rightarrow \text{NULL}$.

WHY DOES `= NULL` GIVE UNKNOWN?

`=` asks whether two *known* values are equal. If either operand is NULL, SQL cannot decide whether they are equal.

`80000 = NULL` → UNKNOWN

`NULL = NULL` → UNKNOWN

WRONG NULL TEST

`WHERE salary = NULL`

Every row produces **UNKNOWN**. Because WHERE retains only **TRUE**, the query returns no rows.

CORRECT NULL TEST

`WHERE salary IS NULL`

Use `IS NULL` or `IS NOT NULL`; these predicates return TRUE or FALSE by testing whether the NULL marker is present.



Three-Valued Logic

Ordinary comparisons involving NULL evaluate to **UNKNOWN** (U). SQL therefore extends TRUE (T) and FALSE (F) to three truth values.

OR

	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

AND

	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

NOT

p	$\neg p$
T	F
F	T
U	U

p IS UNKNOWN tests for U ; WHERE keeps only TRUE, filtering out FALSE and UNKNOWN.

Summarizing Data

Aggregate Functions

Summarize rows with AVG, MIN, MAX, SUM, and COUNT.



Aggregate Functions

Except for `COUNT(*)`, each aggregate summarizes values from one expression and produces one result for the input group.

AVG

average

MIN

minimum

MAX

maximum

SUM

total

COUNT

count

Without `GROUP BY`, the selected rows form one group. With grouping, SQL returns one aggregate result row for each group.



Aggregate Function Examples

Average salary

Comp. Sci. instructors

```
SELECT AVG(salary)
  AS avg_salary
FROM instructor
WHERE dept_name =
  'Comp. Sci.';
```

ONE-ROW RESULT

avg_salary
77333.33

Distinct instructors

Spring 2010

```
SELECT COUNT(DISTINCT ID)
  AS instructor_count
FROM teaches
WHERE semester = 'Spring'
  AND year = 2010;
```

ONE-ROW RESULT

instructor_count
6

Rows in course

```
SELECT COUNT(*)
  AS course_count
FROM course;
```

* is not a column: it means count every input row.

ONE-ROW RESULT

course_count
13

COUNT(*) counts all rows. COUNT(column) counts only rows where that column is non-NULL. Each query above returns a one-row, one-column table.

Grouping and Filtering Groups

Form groups with GROUP BY, filter them with HAVING, and handle NULL in aggregates.



GROUP BY

```
SELECT building, COUNT(*) AS section_count
FROM section GROUP BY building;
```

INPUT: SECTION ROWS

course_id	building
CS-101	Packard
CS-190	Taylor
CS-315	Taylor
BIO-301	Painter

GROUP BY building
+ COUNT(*)



ONE ROW PER GROUP

building	section_count
Packard	1
Taylor	2
Painter	1

The two Taylor rows form one group, so its count is 2; Packard and Painter each form a group with count 1.



GROUP BY: One Value per Group

GROUP BY dept_name must produce one row per department. Every expression in SELECT must therefore have exactly one value for that group.

INPUT ROWS

ID	dept.	salary
10101	CS	65000
45565	CS	75000
83821	CS	92000
12121	Finance	90000
76543	Finance	80000

Invalid in standard SQL

```
SELECT dept_name, ID,  
       AVG(salary)  
FROM instructor  
GROUP BY dept_name;
```

For the CS group, which ID should appear: 10101, 45565, or 83821?
There is no single answer.

Valid: remove ID

```
SELECT dept_name,  
       AVG(salary) AS avg  
FROM instructor  
GROUP BY dept_name;
```

dept_name	avg
CS	77333.33
Finance	85000



Filtering Groups with HAVING

Find departments whose average instructor salary exceeds 42,000.

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

WHERE filter rows



GROUP BY form groups



HAVING filter groups



NULL Values and Aggregates

Total recorded salaries

```
SELECT SUM(salary)
FROM instructor;
```

NULL salaries are ignored.

Expression	Treatment of NULL
COUNT(*)	Counts every row
COUNT(expr)	Counts non-NULL values of expr
SUM / AVG	Ignore NULL input values
MIN / MAX	Ignore NULL input values

If a group has no non-NULL input values, COUNT(expr) returns 0; SUM, AVG, MIN, and MAX return NULL.



Lecture Summary

01

DDL and update commands define tables, enforce constraints, and change data or schemas

02

SELECT-FROM-WHERE, joins, predicates, ordering, and row limits construct and refine queries

03

SQL query results are bags by default; DISTINCT and set operations control duplicate behavior

04

NULL marks missing information; comparisons may yield UNKNOWN, and WHERE keeps only TRUE

05

Aggregates summarize values; GROUP BY forms groups, HAVING filters them, and NULL inputs need care